```
        System.out.println("NOME : GABRIEL AGOSTINHO DA SILVA");
        System.out.println("NOME : ISAQUE DE SOUSA ALMEIDA");
```

```
public class Cell <T>{
    private Cell<T> top;
    private Cell<T> bottom;
    private Cell<T> previus;
    private Cell<T> next;
    private T value;


    public Cell(Cell<T> top, Cell<T> bottom, Cell<T> previus, Cell<T> next, T value){
        this.top = top;
        this.bottom = bottom;
        this.previus = previus;
        this.next = next;
        this.value = value;
    }

    public Cell(){
        this.top = null;
        this.bottom = null;
        this.previus = null;
        this.next = null;
        this.value = null;
    }


    public void setTop(Cell<T> top) {
        this.top = top;
    }

    public void setBottom(Cell<T> bottom) {
        this.bottom = bottom;
    }

    public void setNext(Cell<T> next) {
        this.next = next;
    }
```

```
    public void setPrevius(Cell<T> previus) {
        this.previus = previus;
    }
    public void setValue(T value) {
        this.value = value;
    }

    public Cell<T> getTop() {
        return  top;
    }
    public Cell<T> getNext() {
        return next;
    }

    public Cell<T> getPrevius() {
        return previus;
    }
    public Cell<T> getBottom() {
        return bottom;
    }


    public T getValue() {
        return value;
    }
}



interface AcessCell<T>{
    public abstract T getValue();
    public abstract void setValue(T value);
}

class ManagerCell <T> implements AcessCell<T>{
    Cell<T> cell;
    public ManagerCell(Cell<T> cell){
        this.cell = cell;
    }

    @Override
    public T getValue() {
        return cell.getValue();
    }
```

```java
    @Override
    public void setValue(T value) {
        cell.setValue(value);
    }

}

public class FlexMatriz<T>{
    private Cell<T> sentinel;
    private int columns;
    private int rows;

    public FlexMatriz(int rows, int columns){
        this.columns = columns;
        this.rows = rows;
        sentinel = new Cell<T>();

        if(columns <= 0 || rows <= 0){
            throw new IndexOutOfBoundsException("TAMANHO INVÁLIDO");
        }
        generateMatriz();
    }

    public FlexMatriz(FlexMatriz<T> flex){
        this.columns = flex.getColumns();
        this.rows = flex.getRows();
        sentinel = new Cell<T>();
        generateMatriz();

        for(int i =0; i < rows; i++){
            for(int j =0; j < columns; j++){
                this.at(i,j).setValue(flex.at(i, j).getValue());
            }
        }

    }

    private void generateMatriz(){
        Cell<T> lineAux = sentinel;
        for(int i = 0; i < columns - 1; i++){
            Cell<T> newCell = new Cell<T>();
            lineAux.setNext(newCell);
```

```
        newCell.setPrevius(lineAux);
        lineAux = newCell;
    }

    Cell<T> cursorLine;
    Cell<T> lineOld = cursorLine = sentinel;
    int auxLength = columns == 1 ? 1 : 0;

    for(int i =0; i < rows - 1; i++){
        Cell<T> newLineAux = new Cell<T>();
        for (int j = 0; j < (columns + auxLength) - 1; j++) {
            newLineAux.setTop(lineOld);
            lineOld.setBottom(newLineAux);
            lineOld = lineOld.getNext();

            Cell<T> newCell = new Cell<T>();
            newLineAux.setNext(newCell);
            newCell.setPrevius(newLineAux);
            newLineAux = newCell;
        }

        lineOld = cursorLine = cursorLine.getBottom();
    }
}

public int getColumns() {
    return columns;
}
public int getRows() {
    return rows;
}


public AcessCell<T> at(int line, int column){
    Cell<T> temp = sentinel;

    if(line < 0 || column < 0 || column > columns || line > rows){
        throw new IndexOutOfBoundsException("Valor fora do escopo");
    }

    for(int i = 0; i < line; i++){
        temp = temp.getBottom();
    }
```

```
        for(int j = 0; j < column; j++){
            temp = temp.getNext();
        }

        return new ManagerCell<T>(temp);
    }


    @Override
    public String toString() {
        String returnValue = new String();
        for (int i = 0; i < this.getRows(); i++) {
            for (int j = 0; j < this.getColumns(); j++) {
                    returnValue += "[" + i  + "]" + "[" + j + "]" + "=" + this.at(i,
j).getValue() + "\t";
            }
            returnValue += "\n";
        }
        return returnValue;
    }

}



import java.util.Random;

public class Main {
    public static void soma(FlexMatriz<Integer> flex1 ,FlexMatriz<Integer> otherFlex) throws
Exception{
        if(otherFlex.getRows() != flex1.getRows() || flex1.getColumns() !=
otherFlex.getColumns()){
            throw new Exception("Error os tamanhos diferentes de colunas ou linhas");
        }

        for(int i =0; i < flex1.getRows(); i++){
            for(int  j = 0; j < flex1.getColumns(); j++){
                flex1.at(i, j).setValue(flex1.at(i, j).getValue() + otherFlex.at(i,
j).getValue());
            }
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("NOME : GABRIEL AGOSTINHO DA SILVA");
        System.out.println("NOME : ISAQUE DE SOUSA ALMEIDA");
        FlexMatriz<Integer> matriz = new FlexMatriz<Integer>(8, 8);

        for (int i = 0; i < matriz.getRows(); i++) {
            for (int j = 0; j < matriz.getColumns(); j++) {
                    matriz.at(i, j).setValue(new Random().nextInt(90));
            }
        }

        System.out.println("----------------------------------MATRIZ
1----------------------------------");
        System.out.println(matriz);

        FlexMatriz<Integer> matriz2 = new FlexMatriz<Integer>(matriz);

        try {
            soma(matriz, matriz2);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        System.out.println("----------------------------------MATRIZ
2----------------------------------");
        System.out.println(matriz2);
        System.out.println("----------------------------------SOMA
MATRIZ----------------------------------");
        System.out.println(matriz);

    }
}
```